

Day 1 - JavaScript & Node.js Cơ bản

Mục tiêu: Nắm vững nền tảng JavaScript ES6+ và hiểu tổng quan về Node.js runtime

Sau Day 1, thực tập sinh có thể:

- Phân biệt `var`, `let`, `const` và giải thích khi nào dùng cái nào
- Viết function theo nhiều cách (declaration, expression, arrow function)
- Xử lý bất đồng bộ bằng callback, Promise, async/await
- Giải thích Node.js là gì và event loop hoạt động ra sao
- Tạo module, export và import theo CommonJS và ESM
- Khởi tạo project với npm, hiểu cấu trúc package.json

Câu hỏi tìm hiểu trước

Trước khi học, tìm hiểu và trả lời các câu hỏi sau. Không cần đúng hoàn toàn — mục tiêu là có hình dung ban đầu trước khi đào sâu.

JavaScript là gì?

- JavaScript là gì, chạy ở đâu (browser, server)?
- Sự khác nhau giữa JavaScript và Java?
- JavaScript là ngôn ngữ compiled hay interpreted?

Kiểu dữ liệu

- JavaScript có những kiểu dữ liệu (data type) nào?
- Primitive type và Reference type khác nhau như thế nào?
- `null` và `undefined` khác nhau ra sao?
- Tại sao `typeof null === "object"` ?

Biến & Scope

- Hoisting là gì? `var`, `let`, `const` có bị hoisting không?
- Scope là gì? Phân biệt function scope và block scope?
- Closure là gì, cho ví dụ?
- `var` có những vấn đề gì mà người ta không dùng nữa?

Số & IEEE 754

- Tại sao `0.1 + 0.2 !== 0.3` trong JavaScript?
- `NaN` là gì? `NaN === NaN` trả về gì?
- `Number.MAX_SAFE_INTEGER` là bao nhiêu và tại sao quan trọng?

Tham trị & Tham chiếu

- Primitive và Object được truyền vào function theo cơ chế nào?
- Shallow copy và deep copy khác nhau thế nào?
- Làm sao để so sánh hai object có cùng giá trị?

Bất đồng bộ

- Synchronous và Asynchronous khác nhau như thế nào?
- Event loop trong JavaScript là gì?
- Callback hell là gì, tại sao cần Promise?

- `async/await` có phải là cú pháp mới của Promise không?

Node.js

- Node.js là gì, khác browser JavaScript ở điểm nào?
- Node.js dùng để làm gì, phù hợp với loại ứng dụng nào?
- CommonJS và ES Modules khác nhau như thế nào?
- `npm` là gì, `package.json` dùng để làm gì?

Phần 0 - Setup môi trường

Kiểm tra môi trường trước khi bắt đầu:

```
node --version    # >= 18.x
npm --version     # >= 9.x
git --version
code --version
```

Tạo repo GitHub cá nhân để submit bài tập.

Tài nguyên:

- [Node.js download](#)
- [VS Code](#)

Phần 1 - Biến & Kiểu dữ liệu

var / let / const

```
// var - function scoped, có hoisting, tránh dùng
var x = 1;

// let - block scoped, có thể gán lại
let count = 0;
count = 1; // ok

// const - block scoped, không gán lại được
const PI = 3.14;
// PI = 3; // TypeError
```

Quy tắc thực tế:

- Mặc định dùng `const`
- Dùng `let` khi cần gán lại
- Không dùng `var`

Kiểu dữ liệu

```
// Primitive types
const name = "Alice";           // string
```

```

const age = 25;           // number
const isActive = true;    // boolean
const nothing = null;     // null
let notDefined;           // undefined
const id = Symbol("id");  // symbol

// Reference types
const arr = [1, 2, 3];    // Array
const obj = { key: "value" }; // Object
const fn = () => {};      // Function

```

Kiểm tra kiểu

```

typeof "hello";           // "string"
typeof 42;                 // "number"
typeof true;              // "boolean"
typeof null;              // "object" (bug nổi tiếng của JS)
typeof undefined;        // "undefined"
typeof [];                // "object"
Array.isArray([]);        // true

```

IEEE 754 & Floating Point

JavaScript dùng IEEE 754 double-precision (64-bit) cho tất cả số. Điều này gây ra lỗi làm tròn:

```

0.1 + 0.2 === 0.3          // false!
0.1 + 0.2                  // 0.30000000000000004

// So sánh float dùng epsilon
Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON // true

// Làm tròn an toàn
+(0.1 + 0.2).toFixed(10) === 0.3 // true

// Giữ y với số lớn
Number.MAX_SAFE_INTEGER    // 9007199254740991 (2^53 - 1)
9007199254740992 === 9007199254740993 // true (mất độ chính xác!)
// -> Dùng BigInt cho số nguyên lớn
const big = 9007199254740993n;

```

Quy tắc thực tế:

- Không so sánh float bằng `===`, dùng epsilon hoặc `toFixed`
- Số nguyên lớn hơn `2^53 - 1` -> dùng `BigInt`
- Tính tiền -> nhân số nguyên rồi chia (ví dụ: tính bang xu, hiển thị bang đồng)

Tham trị & Tham chiếu (Pass by Value vs Pass by Reference)

Primitives truyền **giá trị** (copy), objects/arrays truyền **tham chiếu** (địa chỉ):

```
// Pass by value - primitives
let a = 10;
let b = a;
b = 20;
console.log(a); // 10 - a không đổi

// Pass by reference - objects
let obj1 = { x: 10 };
let obj2 = obj1;
obj2.x = 20;
console.log(obj1.x); // 20 - obj1 bị thay đổi!

// Pass by reference - arrays
let arr1 = [1, 2, 3];
let arr2 = arr1;
arr2.push(4);
console.log(arr1); // [1, 2, 3, 4] - arr1 bị thay đổi!
```

Clone object/array để tránh side effect:

```
// Shallow copy
const copy1 = { ...obj1 }; // spread
const copy2 = Object.assign({}, obj1);
const arrCopy = [...arr1];

// Shallow copy chỉ sao chép 1 cấp - nested objects vẫn là tham chiếu
const user = { name: "Alice", address: { city: "HCM" } };
const userCopy = { ...user };
userCopy.address.city = "HN";
console.log(user.address.city); // "HN" - vẫn bị thay đổi!

// Deep copy - an toàn với nested objects (không có function/circular ref)
const deepCopy = JSON.parse(JSON.stringify(user));

// Hoặc dùng structuredClone (Node 17+)
const deepCopy2 = structuredClone(user);
```

So sánh objects:

```
// == và === so sánh địa chỉ, không phải giá trị
{} === {} // false (hai object khác nhau trong bộ nhớ)
[] === [] // false

const a = {};
const b = a;
a === b; // true (cùng một địa chỉ)

// Để so sánh giá trị -> JSON.stringify hoặc deep equal
```

```
JSON.stringify({a: 1}) === JSON.stringify({a: 1}); // true (chú ý: thứ tự key phải giống)
```

Bài tập 1 - exercises/01-variables.js

```
// 1.1: Khai báo và sử dụng biến
// Tạo các biến sau và log kết quả ra console:
// - Họ tên đầy đủ của bạn (const)
// - Tuổi của bạn (let)
// - Có phải lập trình viên không (const)
// - Danh sách kỹ năng của bạn (const array)

// 1.2: Template literals
// Tạo string mô tả bạn thân dùng template literal
// Ví dụ: "Tôi là Nguyễn Văn A, 22 tuổi, là lập trình viên"

// 1.3: Phép toán cơ bản
// Tính: tổng, hiệu, tích, thương của hai số bất kỳ
// Dùng Math.round(), Math.floor(), Math.ceil()

// 1.4: Kiểm tra kiểu
// Viết function nhận vào một giá trị, log ra kiểu dữ liệu
// Xử lý đặc biệt cho null và Array
```

Phần 2 - Function & Array Methods

Các cách viết function

```
// 1. Function Declaration - hoisting, có thể gọi trước khi khai báo
function greet(name) {
  return `Hello, ${name}!`;
}

// 2. Function Expression - không hoisting
const greet2 = function(name) {
  return `Hello, ${name}!`;
};

// 3. Arrow Function - ngắn gọn, không có this riêng
const greet3 = (name) => `Hello, ${name}!`;

// Arrow function nhiều dòng
const greet4 = (name) => {
  const message = `Hello, ${name}!`;
  return message;
};
```

Default parameters & Rest/Spread

```
// Default parameter
function createUser(name, role = "intern") {
  return { name, role };
}

// Rest parameters - gom các argument còn lại thành array
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3, 4); // 10

// Spread - trải array ra thành các argument riêng lẻ
const nums = [1, 2, 3];
Math.max(...nums); // 3
```

Destructuring

```
// Array destructuring
const [first, second, ...rest] = [1, 2, 3, 4, 5];
// first = 1, second = 2, rest = [3, 4, 5]

// Object destructuring
const { name, age, city = "HCM" } = { name: "Alice", age: 25 };
// name = "Alice", age = 25, city = "HCM" (default)

// Trong function parameter
function display({ name, age }) {
  console.log(`${name} is ${age} years old`);
}
```

Array methods quan trọng

```
const users = [
  { name: "Alice", age: 25, active: true },
  { name: "Bob", age: 30, active: false },
  { name: "Charlie", age: 22, active: true },
];

// map - biến đổi từng phần tử, trả về array mới
const names = users.map(u => u.name);
// ["Alice", "Bob", "Charlie"]

// filter - lọc phần tử thỏa điều kiện
const activeUsers = users.filter(u => u.active);

// find - tìm phần tử đầu tiên thỏa điều kiện
const alice = users.find(u => u.name === "Alice");

// reduce - gộp thành một giá trị
```

```
const totalAge = users.reduce((sum, u) => sum + u.age, 0);

// forEach - duyệt, không trả về array mới
users.forEach(u => console.log(u.name));

// some / every
const hasInactive = users.some(u => !u.active); // true
const allActive = users.every(u => u.active); // false
```

Bài tập 2 - exercises/02-functions.js

```
// 2.1: Viet lai cac function sau theo 3 cach (declaration, expression, arrow)
// - Tinh dien tich hinh chu nhat
// - Kiem tra so chan/le
// - Dao nguoc chuoai

// 2.2: Array methods
// Cho mang san pham sau:
const products = [
  { id: 1, name: "Laptop", price: 25000000, category: "tech", inStock: true },
  { id: 2, name: "Phone", price: 15000000, category: "tech", inStock: false },
  { id: 3, name: "Desk", price: 5000000, category: "furniture", inStock: true },
  { id: 4, name: "Chair", price: 3000000, category: "furniture", inStock: true },
  { id: 5, name: "Monitor", price: 8000000, category: "tech", inStock: true },
];

// a) Lay danh sach ten tat ca san pham
// b) Loc san pham con hang (inStock: true)
// c) Loc san pham thuoc category "tech" va con hang
// d) Tinh tong gia tri tat ca san pham con hang
// e) Tim san pham dat nhat
// f) Sap xep san pham theo gia tang dan

// 2.3: Destructuring
// Viet function nhan vao object user { firstName, lastName, age, address: { city, district } }
// Log ra: "Nguyen Van A (25 tuoi) - Quan 1, HCM"
```

Phần 3 - Bất đồng bộ: Callback, Promise, Async/Await

JavaScript chạy single-thread. Nếu phải chờ I/O (đọc file, gọi API), thread bị block. Bất đồng bộ giúp JS tiếp tục chạy code khác trong khi chờ.

Callback - cách cũ

```
// Vấn đề: Callback hell
getUser(id, function(err, user) {
  if (err) return handleError(err);
  getOrders(user.id, function(err, orders) {
```

```

    if (err) return handleError(err);
    getOrderDetails(orders[0].id, function(err, details) {
      if (err) return handleError(err);
      console.log(details); // lồng 3 cấp, khó đọc
    });
  });
});

```

Promise - cải thiện callback

```

// Tạo Promise
const fetchUser = (id) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id > 0) {
        resolve({ id, name: "Alice" }); // thành công
      } else {
        reject(new Error("Invalid ID")); // thất bại
      }
    }, 1000);
  });
};

// Dùng Promise
fetchUser(1)
  .then(user => {
    console.log(user);
    return fetchOrders(user.id);
  })
  .then(orders => console.log(orders))
  .catch(err => console.error(err))
  .finally(() => console.log("Done"));

// Promise.all - chạy song song, chờ tất cả xong
Promise.all([fetchUser(1), fetchUser(2)])
  .then(([user1, user2]) => console.log(user1, user2));

// Promise.race - lấy kết quả nhanh nhất
Promise.race([slowFetch(), fastFetch()])
  .then(result => console.log(result));

```

Async/Await - cách hiện đại (dùng cái này)

```

// async function luôn trả về Promise
async function getUserData(id) {
  try {
    const user = await fetchUser(id);
    const orders = await fetchOrders(user.id);
    return { user, orders };
  }
}

```

```

    } catch (error) {
      console.error("Error:", error.message);
      throw error;
    }
  }

// Chạy song song với async/await
async function loadDashboard(userId) {
  const [user, orders, notifications] = await Promise.all([
    fetchUser(userId),
    fetchOrders(userId),
    fetchNotifications(userId),
  ]);
  return { user, orders, notifications };
}

```

Bài tập 3 - exercises/03-async.js

```

// 3.1: Chuyển đổi callback sang Promise
// Viết lại hàm sau dùng Promise thay vì callback:
function readFileFake(filename, callback) {
  setTimeout(() => {
    if (filename.endsWith(".txt")) {
      callback(null, `Content of ${filename}`);
    } else {
      callback(new Error("Only .txt files allowed"));
    }
  }, 500);
}

// 3.2: Chuỗi Promise
// Tạo 3 hàm giả lập API call (mỗi hàm dùng setTimeout 500ms):
// - getUser(id) -> trả về { id, name, teamId }
// - getTeam(teamId) -> trả về { id, teamName, managerId }
// - getManager(managerId) -> trả về { id, name, email }
// Dùng Promise chain để lấy thông tin manager của user có id = 1

// 3.3: Async/Await với error handling
// Viết lại Bài 3.2 dùng async/await
// Thêm: nếu id <= 0 thì throw Error("Invalid user ID")
// Đảm bảo lỗi được xử lý đúng cách

// 3.4: Promise.all vs tuần tự
// So sánh thời gian chạy:
// a) Gọi getUser(1), getUser(2), getUser(3) tuần tự (await từng cái)
// b) Gọi Promise.all([getUser(1), getUser(2), getUser(3)])
// Log thời gian của mỗi cách

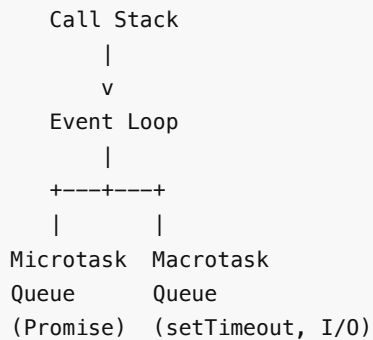
```

Phần 4 - Node.js Overview & Module System

Node.js là gì?

- Runtime cho JavaScript bên ngoài browser (dựa trên V8 engine của Chrome)
- Non-blocking I/O, event-driven
- Single-threaded nhưng xử lý concurrent nhờ event loop
- Phù hợp: API server, real-time app, CLI tools, microservices

Event Loop



Thứ tự ưu tiên:

1. Call Stack (code đang chạy)
2. Microtask Queue (Promise.then, queueMicrotask)
3. Macrotask Queue (setTimeout, setInterval, I/O callbacks)

```
console.log("1");
setTimeout(() => console.log("2"), 0); // Macrotask
Promise.resolve().then(() => console.log("3")); // Microtask
console.log("4");
// Output: 1, 4, 3, 2
```

CommonJS (require/module.exports)

```
// math.js
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;
module.exports = { add, subtract };
```

```
// app.js
const { add, subtract } = require("./math");
const path = require("path");
const fs = require("fs");
```

ES Modules (import/export)

```
// math.js - cần "type": "module" trong package.json hoặc dùng .mjs
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
```

```
export default { add, subtract };

// app.js
import { add, subtract } from "./math.js";
import math from "./math.js";
import * as MathUtils from "./math.js";
```

	CommonJS	ES Modules
Extension	.js (mặc định)	.mjs hoặc "type":"module"
Sync/Async	Sync (require)	Async (import)
Trong Node.js	Mặc định	Cần config
Dùng khi	Project cũ, đơn giản	Project mới, frontend

npm & package.json

```
npm init -y
npm install express
npm install --save-dev jest
npm install -g nodemon
npm uninstall express
npm run dev
```

```
{
  "name": "my-project",
  "version": "1.0.0",
  "scripts": {
    "start": "node src/index.js",
    "dev": "nodemon src/index.js",
    "test": "jest"
  },
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.0",
    "jest": "^29.0.0"
  }
}
```

Bài tập 4 - exercises/04-nodejs/

```
// 4.1: Event Loop
// Du đoán output trước khi chạy, sau đó chạy để kiểm tra:
console.log("Start");
setTimeout(() => console.log("Timeout 1"), 0);
```

```
setTimeout(() => console.log("Timeout 2"), 100);
Promise.resolve().then(() => console.log("Promise 1"));
Promise.resolve().then(() => {
  console.log("Promise 2");
  setTimeout(() => console.log("Timeout 3"), 0);
});
console.log("End");

// 4.2: CommonJS Module
// Tao module calculator.js voi cac ham: add, subtract, multiply, divide
// divide phai throw Error neu chia cho 0
// Import va test trong index.js

// 4.3: Built-in modules
// Dung module 'fs' va 'path':
// - Doc file hien tai va log so dong
// - Tao file output.txt chua timestamp hien tai
// - Dung path.join() de tao duong dan cross-platform

// 4.4: npm project
// Khoi tao npm project moi trong thu muc day1-npm/
// Cai lodash, viet code dung _.groupBy de nhom mang products theo category
```

Bài tập về nhà - homework/todo-app.js

Xây dựng ứng dụng quản lý TODO chạy trên terminal:

Yêu cầu:

- Lưu data trong mảng (in-memory)
- Các chức năng:
 - `addTodo(title, priority)` - thêm todo mới
 - `completeTodo(id)` - đánh dấu hoàn thành
 - `deleteTodo(id)` - xóa todo
 - `listTodos(filter)` - liệt kê (filter: 'all' , 'active' , 'completed')
 - `searchTodos(keyword)` - tìm kiếm theo title
- Mỗi todo có: `id` (auto-increment), `title` , `priority` ('low'/'medium'/'high'), `completed` (boolean), `createdAt` (Date)
- Dùng `async/await` cho tất cả operations (mô phỏng delay 200ms)
- Có error handling đầy đủ

Bonus: Lưu data vào file JSON dùng module `fs` , đọc lại data từ file khi khởi động

Tiêu chí đánh giá

Tiêu chí	Mô tả	Điểm
Bài 1 - Biến & Types	Code chạy đúng, dùng const/let hợp lý	10
Bài 2 - Functions	Array methods dùng đúng, không dùng vòng lặp thủ công	20

Bài 3 - Async	Promise/async/await đúng, có error handling	25
Bài 4 - Node.js	Module system đúng, built-in modules hoạt động	20
Homework - TODO app	Đủ chức năng, code clean, có async	25

Pass Day 1: ≥ 70 điểm

Tài liệu tham khảo

Bắt buộc đọc:

- [MDN - var/let/const](#)
- [MDN - Arrow functions](#)
- [MDN - Promise](#)
- [MDN - async/await](#)
- [Node.js - Introduction](#)

Đọc thêm:

- [JavaScript.info - Promises, async/await](#)
- [Node.js Event Loop](#)
- [CommonJS vs ES Modules](#)

Video:

- [JavaScript Event Loop - Jake Archibald JSConf](#)

Stuck quá 15 phút với một bài -> hỏi mentor ngay.